

Jane Street Real-Time Market Data Forecasting

Models and Angles: A Technical Report

Alberto Gennaro

July 8, 2026

Abstract

We document the modelling approaches taken on the Jane Street Real-Time Market Data Forecasting task: predicting the weighted- R^2 of `responder_6` from 79 anonymized features streamed per `(symbol, date, time)`. We first summarise what the target *is* — a forward-shifted moving average of a near-white-noise signal, which caps achievable skill — and then describe the leakage-safe data pipeline, the twelve models we tried across five architectural families, the signature and covariate ideas, and the ensembling strategy. The best result to date is a three-model bagged average scoring $R_w^2 = +0.01039$ on a held-out 100-day tail, versus the $+0.0112$ that took 8th place. The central empirical lesson is that, because the target is dominated by noise, *variance reduction* (online refitting, temporal bagging, ensembling) buys more than any single stronger model.

1 Problem and the nature of the target

Each row is indexed by `(symbol_id, date_id, time_id)` with 79 features and 9 responders; we predict `responder_6`, scored by the weighted, zero-mean R^2

$$R_w^2 = 1 - \frac{\sum_i w_i (\hat{y}_i - y_i)^2}{\sum_i w_i y_i^2}.$$

What the responders are. Autocorrelation and Fourier analysis (Kaggle discussion #555562, corroborated by our own tag analysis) show the responders are *moving averages of a few underlying signals*, shifted forward in time: `responder_6` is an SMA of window 20, `responder_7` of window 120, `responder_8` of window 4, of one signal (“exchange A”); responders $\{3, 4, 5\}$ are the same windows of a second signal (“exchange B”); and $\{0, 1, 2\}$ are their differences. The label at time t is effectively the SMA-20 over $[t, t+20]$. The practical consequence is stark: for a forward moving average of near-white noise, $\mathbb{E}[\text{future} \mid \text{present}] \approx 0$, so the predictable component is tiny. This explains why all scores, ours included, plateau near $+0.010$, and it motivates our emphasis on variance reduction over single-model capacity.

Deployability constraint. The evaluation gateway streams one `time_id` at a time and exposes responders only as *daily lags* (the previous day’s full path). Hence (i) any feature used at time t may look back only to $\leq t$ within the day, and (ii) the target’s own history is available only as yesterday’s lagged responders. Every model we build respects this; we verify it explicitly.

2 Data pipeline and leakage control

- **Date range.** The per-day time grid stabilises at 968 steps from `date_id` 677; following the 8th-place solution we train from date 700. Training pool: 700–1598; held-out validation tail: 1599–1698.
- **Features.** Raw features; per-symbol trailing rolling mean/std (window 1000); cross-sectional market average per `(date, time)`; two synthetic auxiliary responders; optionally previous-day lagged responders and signals derived from them.

- **Leakage audit.** Model inputs are trailing or contemporaneous (no forward look). The auxiliary targets use forward shifts by construction; their only cross-boundary reach is removed by a one-day *embargo* between the pool and the validation tail. Standardization statistics are fit on training dates only; the online scaler adapts on past days only. Lagged responders are strictly previous-day (a `date+1` self-join), verified backward-only.
- **Engineering for scale.** Features are pre-computed once for the whole pool into a `Float16` memory-mapped array; each model then reads only the dates it trains on. This is what makes on-laptop training of many models on hundreds of dates feasible.

3 Models and the angle behind each

We treat one (`symbol`, `day`) as one training example: a sequence of 968 timesteps. Predictions are per-timestep and strictly causal within the day.

3.1 Gradient boosting (`xgb`)

A per-row `XGBRegressor` on the flat feature matrix — no time structure. Cheap, strong, and a natural ensemble base because its errors are uncorrelated with the sequence models. Best single static score $\approx +0.0068$.

3.2 Feed-forward baselines (`mlp`)

A plain MLP on the per-row features, as a sanity floor. It scores *negative* (-0.0026): the signal here is temporal, and a model with no access to the path cannot recover it. Useful precisely as a negative control.

3.3 Recurrent models and online refitting (`gru`, `lstm`, `*modelr`)

GRU/LSTM encoders over the daily sequence. The `modelr` variants replicate the 8th-place design: four parallel recurrent branches plus auxiliary heads that also predict responders 7–10, a multi-task regulariser. The decisive ingredient is *online refitting*: at inference the model takes one gradient step per day on the previous day’s realised responders (using the daily lags), adapting to drift. This lifts RNN scores by $+0.003$ to $+0.005$ and is the single most valuable trick we found. `lstm_modelr` with refit learning rate 10^{-3} is our best single model at $R_w^2 = +0.0092$.

3.4 Volterra signatures (`sig_transformer`, `mlp_sig`)

Motivated by rough-path theory, we augment models with the truncated *Volterra signature* of a small set of channels over a rolling within-day window: increments are weighted by a power-law kernel $(T - s)^{H-1/2}$ (Hurst $H < 1/2$ emphasises recent moves), and the truncated tensor algebra is computed by Chen iteration. We explored (i) classical vs *log*-signatures — the latter live in the free Lie algebra and are lower-dimensional and less redundant; the minimal Lyndon-basis log-signature (via `iisignature`) cuts depth-3 dimension from 819 to 285; and (ii) *channel selection*: an $\text{AR}(p)$ ridge fit scoring how well each feature’s own lags predict the target, which cleanly beat a naive univariate-correlation filter. A stateless per-timestep MLP fed the minimal log-signature reached $+0.0066$ online — competitive with the transformer while far simpler — but signature streams never improved the ensemble, being too correlated with the RNNs.

3.5 Transformers: vanilla, inverted, and cross-attention (`transformer`, `itransformer`, `timexer`)

A causal attention encoder is the baseline. We then adapted two recent ideas:

- **iTransformer** (inverted attention): attention runs *across features* rather than time, modelling cross-feature correlations directly. We make it causal by applying the inverted attention per timestep and mixing time with a separate causal block.

- **TimeXer** (exogenous covariates): separate an *endogenous* target series from *exogenous* covariates, with a learnable global token bridging them by cross-attention. In our mapping the endogenous stream is *yesterday’s responder path* (fully known at day start, hence patchable non-causally) and the exogenous stream is today’s features (causal). Because turning today’s features into whole-window variate tokens would leak the future, we invert the bridge direction relative to the paper — the causal today-stream queries the fully-known yesterday-stream. Causality is verified by a perturbation test.

3.6 Selective state space (mamba)

A pure-PyTorch Mamba block — data-dependent linear dynamics with an unbounded latent, a third architectural family distinct from gated RNNs and attention. Included primarily as a source of ensemble diversity.

4 Feature-engineering angles

Beyond the baseline set we investigated: (i) **lagged responders** — yesterday’s full responder path as input features, which is both what the gateway provides and the natural “endogenous” signal; (ii) **responder signals** derived from them — multi-scale momentum (differences across the SMA-4/20/120 windows) and the cross-exchange spread, motivated directly by the reverse-engineered structure; and (iii) **tag-group structure** from `features.csv`, where matched feature triples differing only in tags {12,13,14} behave like three views of one quantity, suggesting level/slope aggregates. Each is added as an opt-in group and validated by ablation before adoption.

5 Ensembling and temporal bagging

Given the noise ceiling, combining models matters more than any single one. A plain average of `xgb` + `gru_modelr`(online) + `lstm_modelr`(online) scores $R_w^2 = +0.01039$, about 93% of the 8th-place mark, and beats both a convex weight-optimised blend and a non-linear XGB stacker (which overfit the fitting split). Adding signature or transformer streams did *not* help — they are too correlated with the RNNs. The response is *temporal bagging*: train the same architecture on different overlapping date resamples (subsampling $\sim 60\%$ of the pool per member, different seeds), producing decorrelated members whose average a simple mean can exploit. This is now the primary lever.

6 Results

Model (280 train dates unless noted)	static R_w^2	online R_w^2
<code>lstm_modelr</code> (refit lr 10^{-3})	+0.00637	+0.00920
<code>gru_modelr</code>	+0.00763	+0.00893
<code>xgb</code>	+0.00684	—
<code>mlp_sig</code> (minimal log-sig, AR channels)	+0.00614	+0.00657
<code>sig_transformer</code> (AR ch., refit 3×10^{-5})	+0.00594	+0.00623
<code>mlp</code> (non-sequence control)	−0.00258	—
3-model simple-average ensemble	+0.01039	

Table 1: Weighted- R^2 on the 1599–1698 validation tail. 8th-place private LB was +0.0112.

Selected findings. (1) More training data helped every model, whereas doubling model width did not — the bottleneck is data and noise, not capacity. (2) Online refit is the RNN lever;

transformers barely benefit and signature-transformers need a $30\times$ smaller refit LR. (3) A static-XGB ablation (200 train days) found that adding the nine lagged responders as *flat* features did not help ($+0.00594 \rightarrow +0.00591$), but the five *engineered* responder signals — multi-scale momentum and the venue spread, differences across the SMA-4/20/120 windows — added $+0.00030$ ($+0.00594 \rightarrow +0.00624$, a $\sim 5\%$ relative gain). The interpretation is instructive: a tree cannot cheaply synthesise cross-window differences from the raw lags, so supplying them explicitly pays off, whereas the raw lags on their own are near-useless to a static model — consistent with the noise ceiling. The lags are expected to matter more for the online RNNs (daily adaptation) and TimeXer’s structured endogenous stream.

XGB feature set (200 train days)	#feat	R_w^2
base	125	+0.00594
+ lagged responders (raw)	134	+0.00591
+ responder signals (engineered)	139	+0.00624

Table 2: Responder feature-engineering ablation. Raw lags are flat; the engineered multi-scale/spread signals give a real lift.

7 Roadmap

In priority order: (1) temporal bagging of the RNN models at scale; (2) confirm whether lagged responders help the *online* models and TimeXer even though they did not help static XGB; (3) a TimeXer vs. flat-lags head-to-head; (4) push the training pool to the full clean range on GPU; (5) multi-seed single-architecture ensembles; (6) per-architecture refit-LR schedules. De-prioritised: larger single models, non-linear stacking, and further signature-channel tuning, all of which showed diminishing returns.